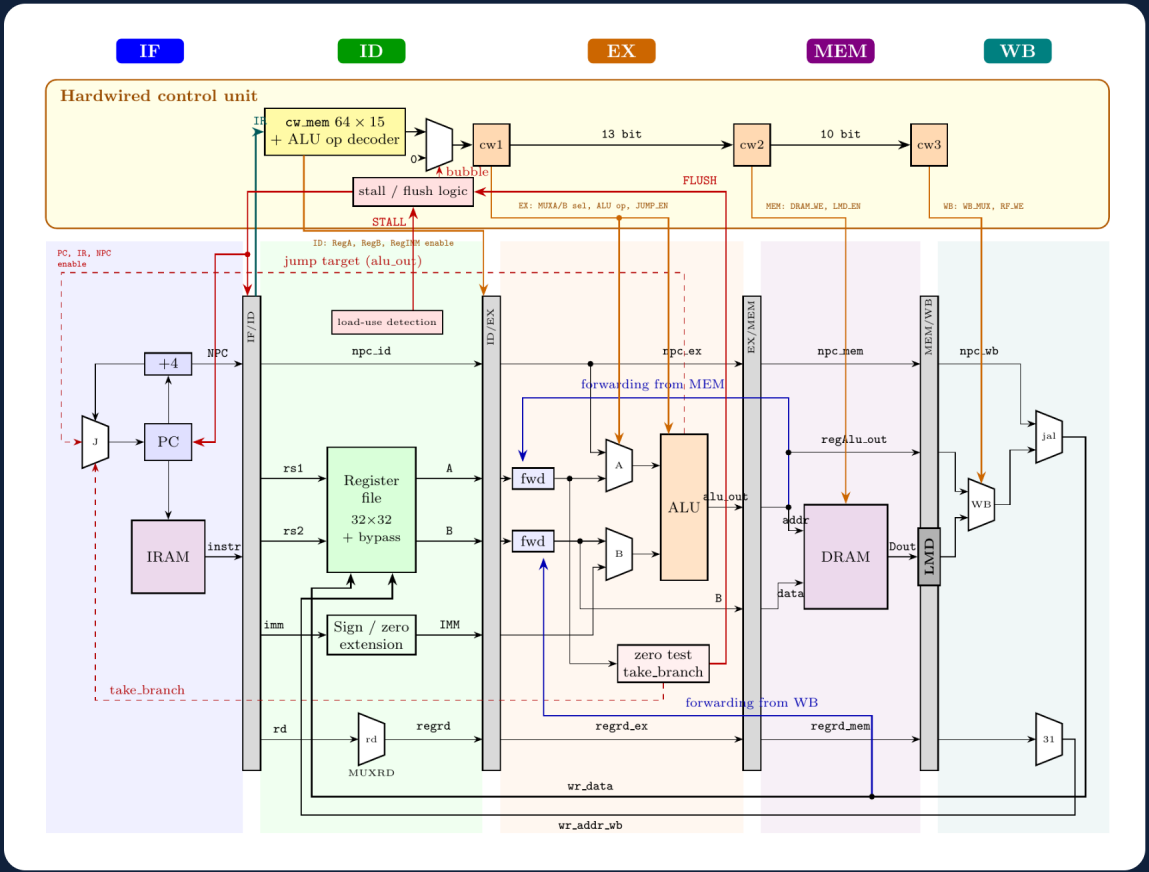
MICROELECTRONIC SYSTEMS · FINAL PROJECT

DLX Microprocessor

A five-stage pipelined DLX pro, from VHDL to layout



Lorenzo Pavone

s362884 · Group 25

Politecnico di Torino

THE PROJECT

Where I started

The assignment

- Choose and complete a CU
- 27 instructions (DLX basic)
- Datapath RTL
- Synthesis and place & route
- Report and commented programs

Provided material

- Package, top level, IRAM
- 3 partial CUs: HW, FSM, µcode
- Empty datapath file
- Testbench, example programs
- dlxasm.pl, conv2memory

What I built

- The whole pipelined datapath
- Control words per stage
- Flush, stall, forwarding
- +23 instructions, P4 ALU
- Synthesis and layout scripts

Key point · from a skeleton with fetch and a control table to a complete processor carried down to the layout.

THE PROJECT

What I implemented

Point	Requirement
1	Five-stage pipeline, hardwired CU, stalls shown
2	27 instructions, ALU opcode as enumeration, commented programs
3	Datapath at RT level
4	Synthesis of CU and datapath, frequency optimization, commented script
5	Place & route: delay, power (EM, thermal: tool not available)
6	Report
7 · pro	+23 instructions, jr jalr lhi included
8, 15 · pro	P4 adder in the ALU, forwarding, critical path
14 · pro	Clock tree synthesis

Key point · all the mandatory points 1–6, plus the pro points 7, 8, 14 and 15: a DLX pro.

RESULTS

What was built

50
instructions
27 basic + 23 extended

2 / 1
cycles lost
2 per taken branch, 1 per load-use

588
MHz
verified after layout, zero violations

15.1k
µm² of cells
chip of 176 × 175 µm, 0.031 mm², NanGate 45 nm

Key point · a basic DLX extended with forwarding, 23 instructions and a P4 adder, verified and carried to layout.

THE PROJECT

How the design grew, step by step

Basic

1. **Basic instruction set**: hardwired CU table and datapath
2. **Provided material** fixed: IRAM addressing, conv2memory
3. **Hazards**: branches in EX with a two-instruction flush, stalls on dependencies
4. **Synthesis** and **place & route**, with frequency optimization

Pro

5. **Forwarding** and RF bypass: only load-use stalls
6. **+23 instructions**: comparisons, shifts, unsigned, jr, jalr, lhi
7. **P4 adder** in the ALU
8. **Clock tree synthesis** in the layout

Key point · each step was verified with its own test program before moving to the next.

INSTRUCTION SET

50 instructions

Basic · 27

```
add sub and or xor
sll srl sne sle sge
addi subi andi ori xori
slli srli snei slei sgei
lw sw beqz bnez j jal nop
```

Extended · 23

```
seq slt sgt + immediate
sltu sgtu sleu sgeu + imm.
sra srai lhi
addu subu addui subui
jr jalr
```

Watch the sign

−3 < 5: slt = 1, sltu = 0

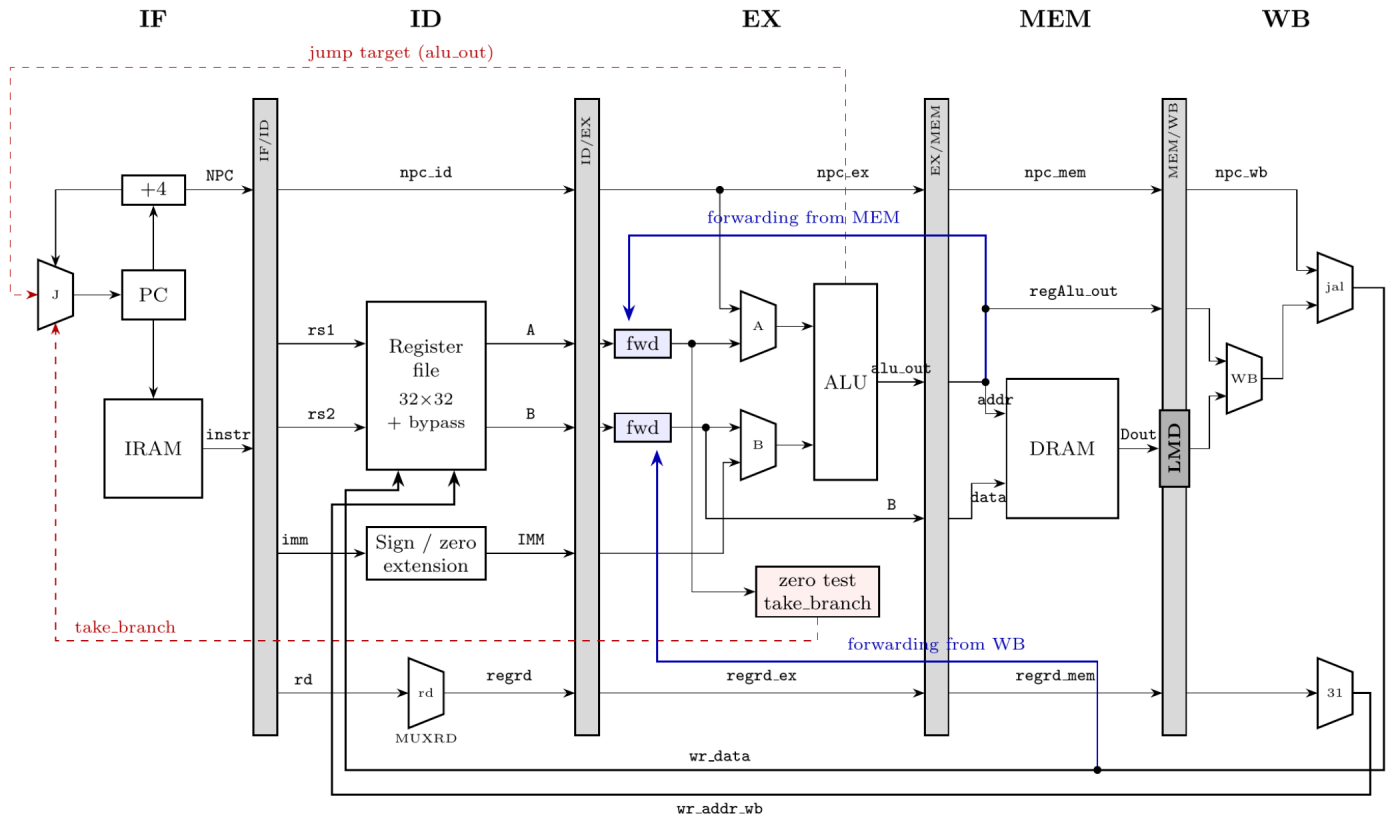
−3 >> 1 = −2 with sra

zero-extended immediates for logical and unsigned

Key point · jr r31 returns from a jal call; lhi + ori build any 32-bit constant.

ARCHITECTURE

The pipelined datapath



Grey bars: pipeline registers between the five stages

Blue: forwarding from MEM and WB to EX

Red: branch target and decision back to the PC

WB: data and r31 for jal back to the register file

ARCHITECTURE

In order, single issue

One per cycle

At most one instruction is fetched, and one completes, per cycle.

Program order

Every instruction reads in ID and writes in WB, in program order.

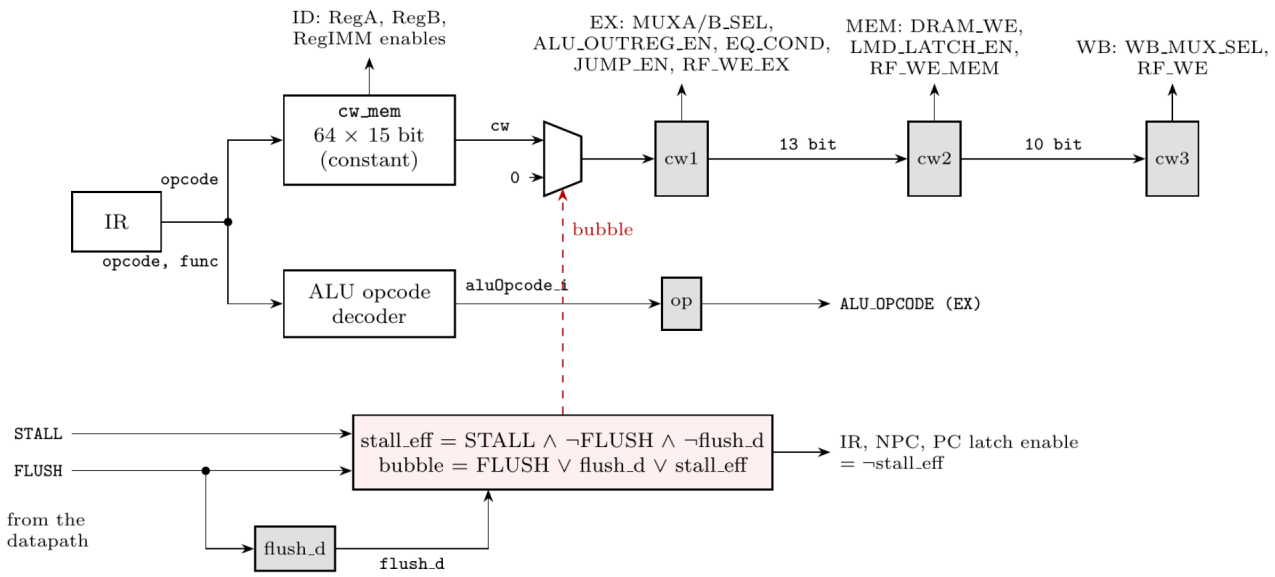
Only RAW

WAR and WAW hazards cannot happen: no renaming, no reorder buffer.

Latency 5 cycles, throughput 1 instruction per cycle without hazards.

Key point · the hazard logic only compares the sources of the younger instruction with the destinations of the older ones.

A table and three registers



64 × 15 constant table, indexed by the opcode

ALU operation decoded apart, as an **enumerated type**

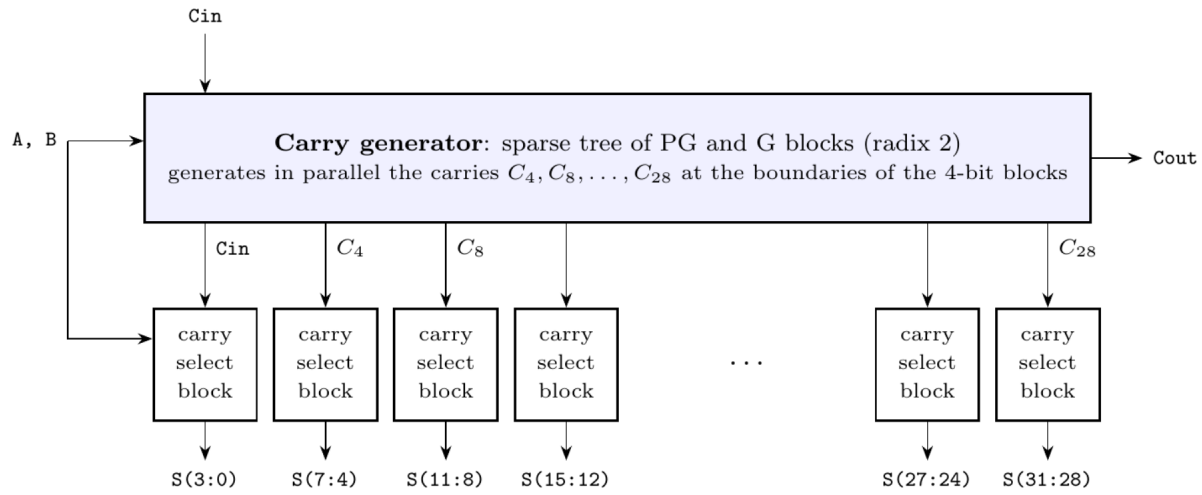
A bubble is a **zero** control word in cw1

21 flip-flops after synthesis: cw1 9 + cw2 4 + cw3 2 + ALU op 5 + flush_d 1

Key point · each control word travels with its instruction: cw1 drives EX, cw2 MEM, cw3 WB.

ARCHITECTURE

The ALU adder: the P4 of the labs



Sum generator: 8 carry-select blocks of 4 bits, each with 2 RCAs and a mux

Carry generator: sparse tree, carries every 4 bits in log levels

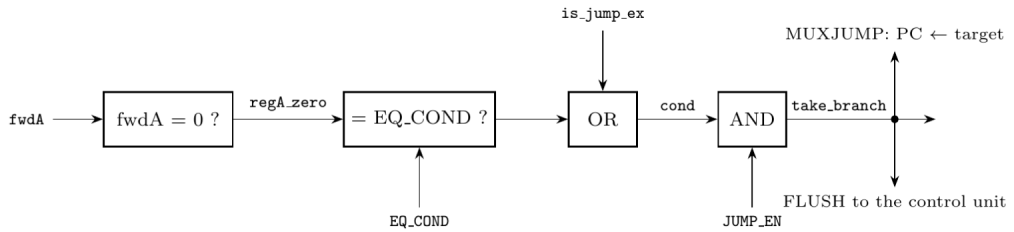
Sum generator: carry-select blocks, both sums precomputed

One adder for + and −: A − B = A + not(B) + 1

542 µm² after layout, 3.6% of the cell area

Key point · for a subtraction the ALU inverts B and sets Cin = 1: the same P4 does both operations.

Branches are resolved in EX

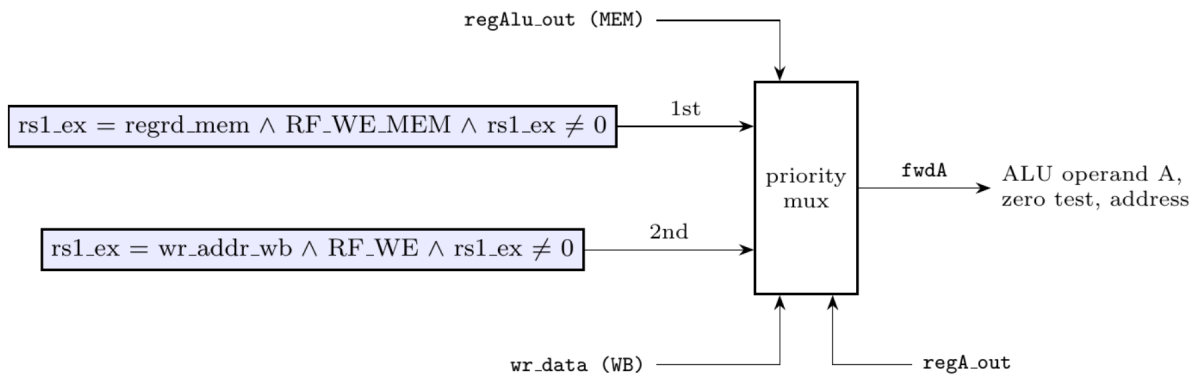


Instruction	c1	c2	c3	c4	c5
beqz r0, l1	IF	ID	EX	MEM	WB
addi r27,r0,#99		IF	ID	flush	
addi r27,r0,#98			IF	flush	
l1: bnez r25,l2				IF	ID

Instructions from test_all.asm

Key point · a taken branch or jump costs 2 cycles; the zero test uses the forwarded value, so no stall.

Forwarding covers every distance



From MEM

distance 1: ALU result of the previous instruction, regAlu_out

From WB

distance 2: value being written back, wr_data

Register file bypass

distance 3: WB writes and ID reads the same register

Load-use

the stall logic inserts one bubble, then the loaded data arrives from WB

Key point · MEM has priority over WB because it holds the most recent value; r0 is never forwarded.

Forwarding and stall in time

No stall

Instruction	c1	c2	c3	c4	c5	c6
addi r1,r0,#12	IF	ID	EX	MEM	WB	
addi r2,r0,#5		IF	ID	EX	MEM	WB
add r3,r1,r2			IF	ID	EX	MEM

add needs r2, written just before, and r1, written two before: in c5 it takes r2 from MEM and r1 from WB, with no lost cycle.

Load-use: one bubble

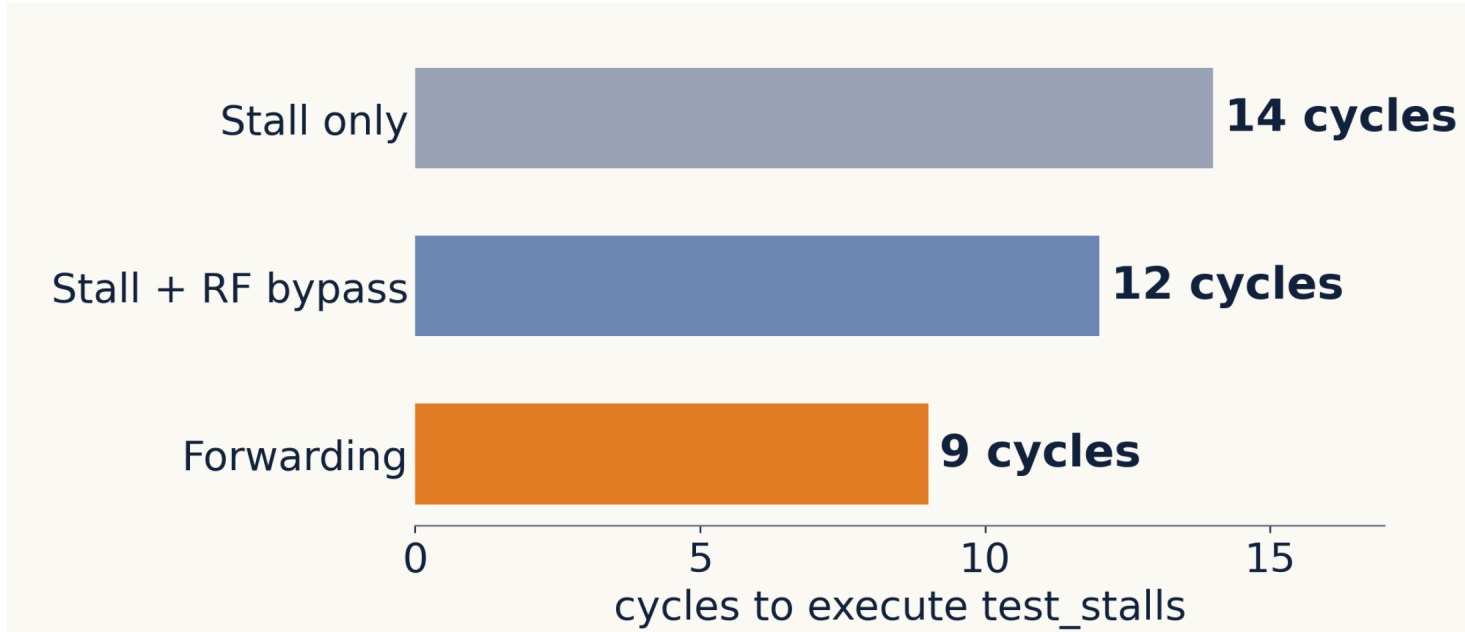
Instruction	c1	c2	c3	c4	c5	c6
lw r24,0(r0)	IF	ID	EX	MEM	WB	
bubble				EX	MEM	WB
add r24,r24,r1		IF	ID	ID	EX	MEM

add needs r24 from the load just before: it stays in ID one more cycle while a bubble fills EX, then takes r24 from WB.

Instructions from test_all.asm

Key point · the bubble is the empty slot that stays in EX while the add waits in ID.

Same program, fewer cycles



Same program, **same results**

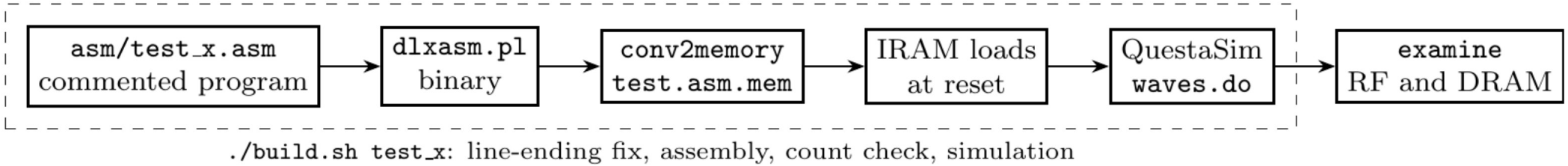
Last write: 31 ns → 21 ns in simulation

Cost: forwarding muxes on the **critical path**

Key point · 14 → 9 cycles on test_stalls just by forwarding values instead of waiting.

VERIFICATION

Each test leaves a visible trace



Program	Checks
test_alu / cmp	ALU, signed compare
test_mem	sw, lw, negative values
test_jump / jal / jr	flush, r31, calls
test_branch / loop	every branch, a loop over memory
test_stalls	forwarding, load-use
test_ext / lhi	the new instructions
test_all	everything, 48 instructions

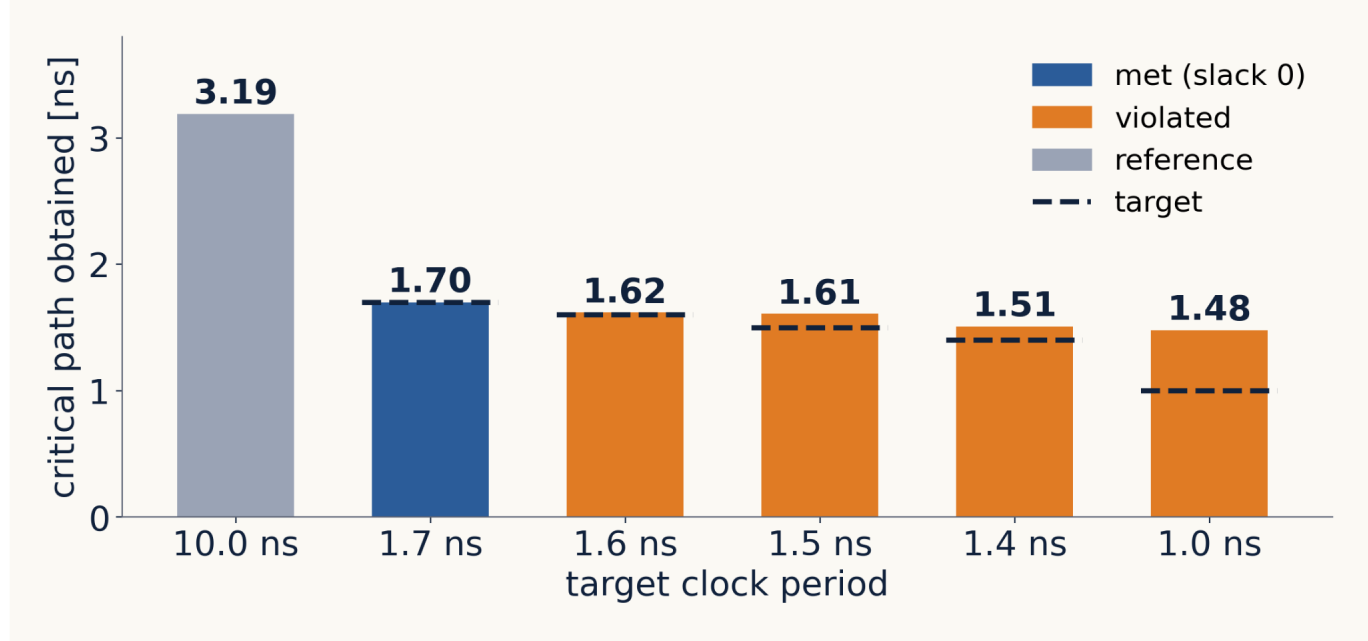
Flushed instructions all write **r27**

r27 = 0 at the end proves every flush

test_all fills the IRAM and checks all 32 registers

SYNTHESIS

From 313 to 676 MHz



DLX_SYN: control unit + datapath, memories excluded

Reference: 3.19 ns

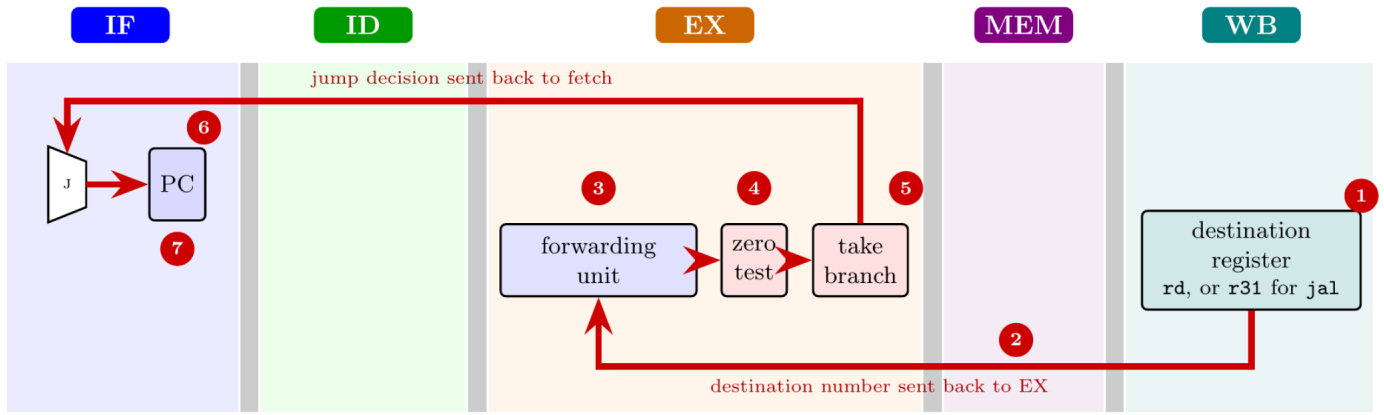
Fastest: **1.48 ns** ≈ 676 MHz

Met: **1.7 ns** ≈ 588 MHz, used for layout

Key point · over-constraining gives the fastest circuit; the tool only works as hard as the constraint asks.

SYNTHESIS

The critical path is pipeline control



Why

A value from WB is forwarded, used by the ALU or the zero test, decides the jump and selects the next PC, in one cycle.

Consequence

It sets the maximum frequency. A faster adder would not help: the fix is in the control, e.g. computing the forwarding comparisons one stage earlier, in ID.

Key point · forwarding + branch in EX save cycles but form the longest path of the processor.

LESSON LEARNED

Correct in simulation, empty in silicon

```
signal cw_mem : mem_array := ( ... );
```

1 · Synthesis

Design Compiler ignores initial values: every CU output tied to 0

2 · Hidden

no constant propagation across hierarchy: the reports looked fine

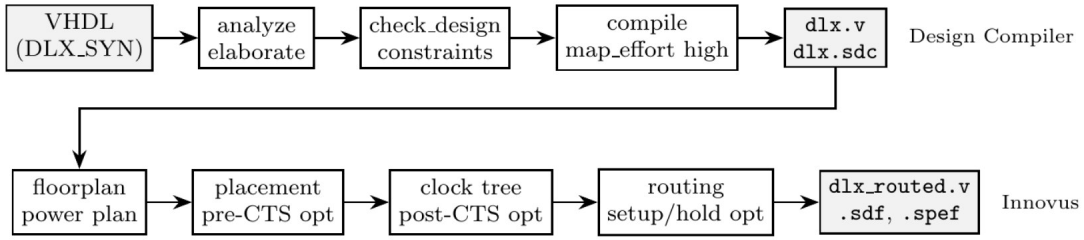
3 · Layout

Innovus flattens and removes 1 045 flip-flops, register file included

Key point · fix: constant cw_mem, and check_design in every synthesis run.

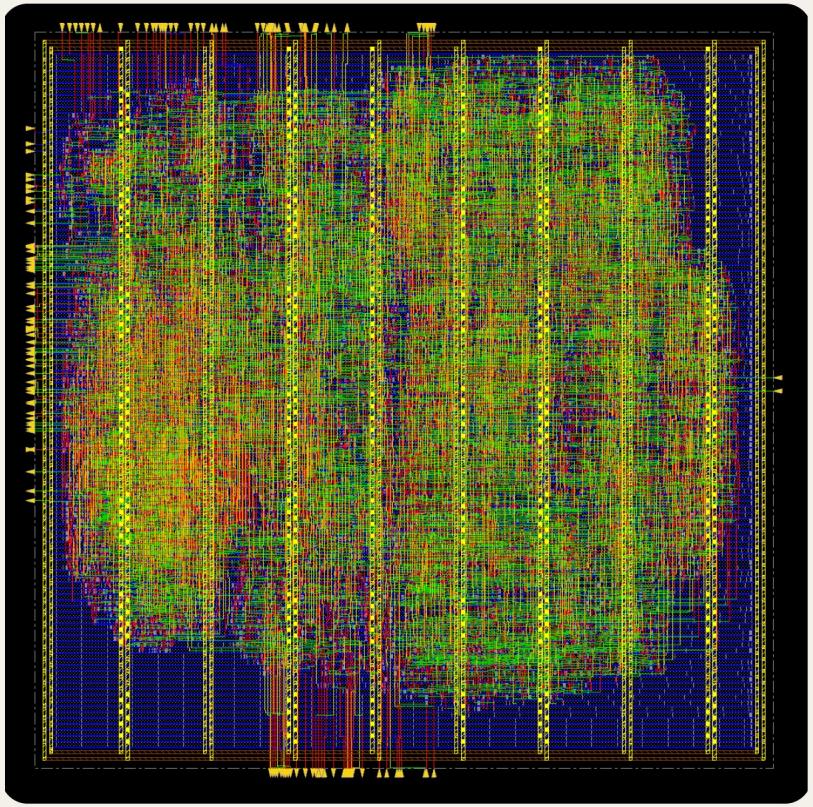
PHYSICAL DESIGN

Clean at 588 MHz after layout



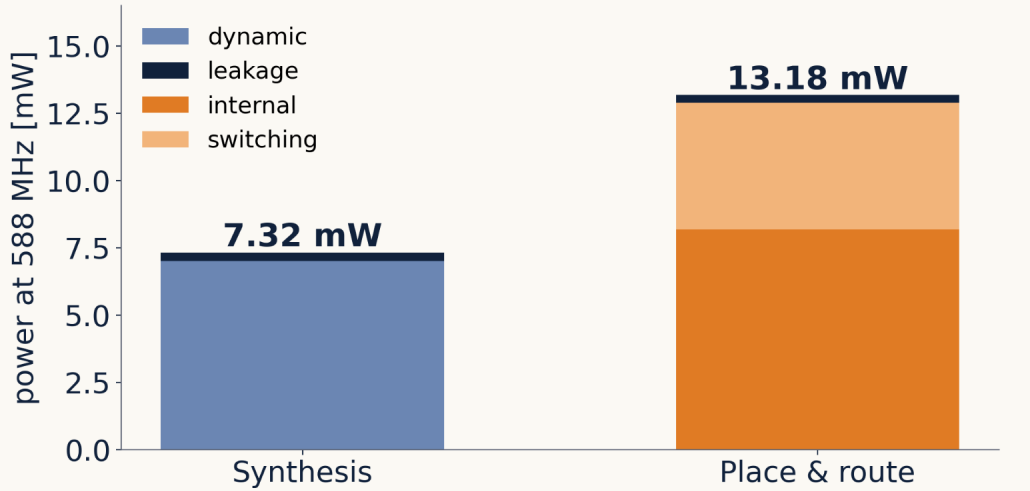
Check	Result
Setup WNS	+0.165 ns
Hold WNS	0.000 ns
Violations, DRV	0
Cell area	15 094 μm²
Die	176 × 175 μm

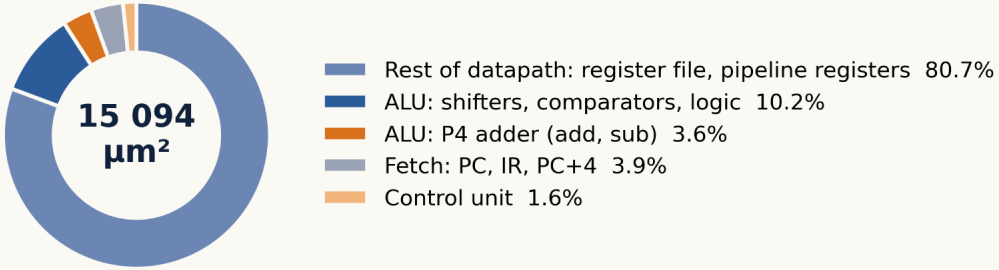
Key point · slack +0.165 ns ≈ 650 MHz; filler cells go last, or the hold buffers find no room.



PHYSICAL DESIGN

Power and area





Key point · power almost doubles after layout (clock tree, real wires); ALU: 13.8% of the cell area.

CONCLUSIONS

What could come next

Branch penalty

Branch decided in ID, or a branch history table: from 2 lost cycles to 1 or 0

Memory instructions

lb, lbu, lh, lhu, sb, sh: byte selection in MEM

Multiplier

mult with the Booth multiplier of the labs, on more cycles

Control word

remove the bits no stage reads any more

Post-layout simulation

run the programs on the netlist with the SDF delays

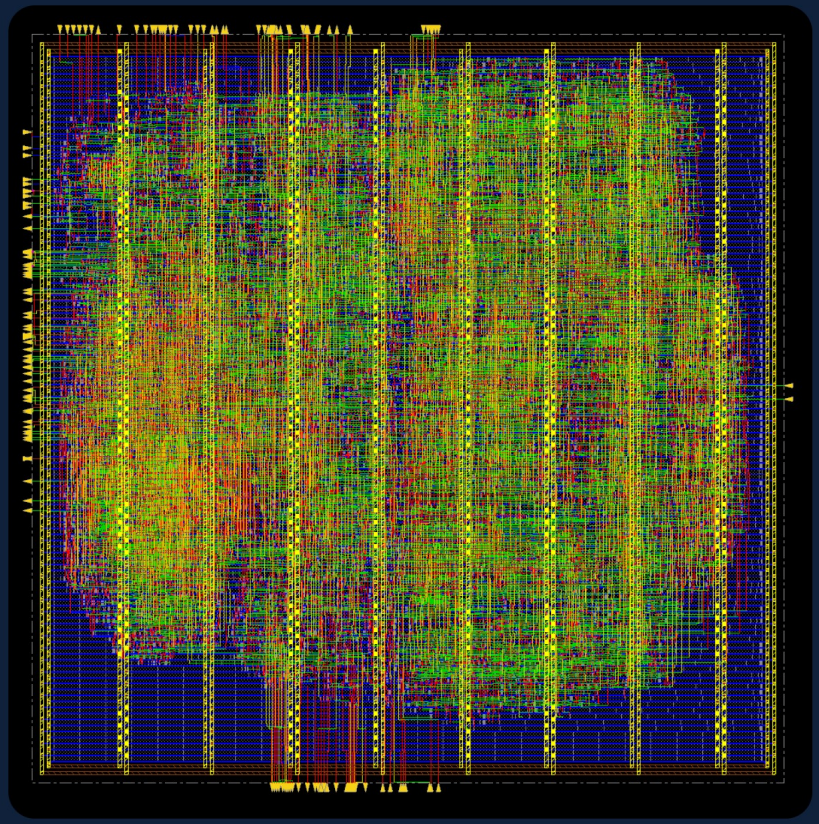
Memory interface

input and output delays for a realistic timing of the ports

Key point · the critical path is in the pipeline control: every improvement must be weighed against the cycle time.

DLX MICROPROCESSOR

Thank you for your attention



Lorenzo Pavone

s362884 · Group 25

Politecnico di Torino